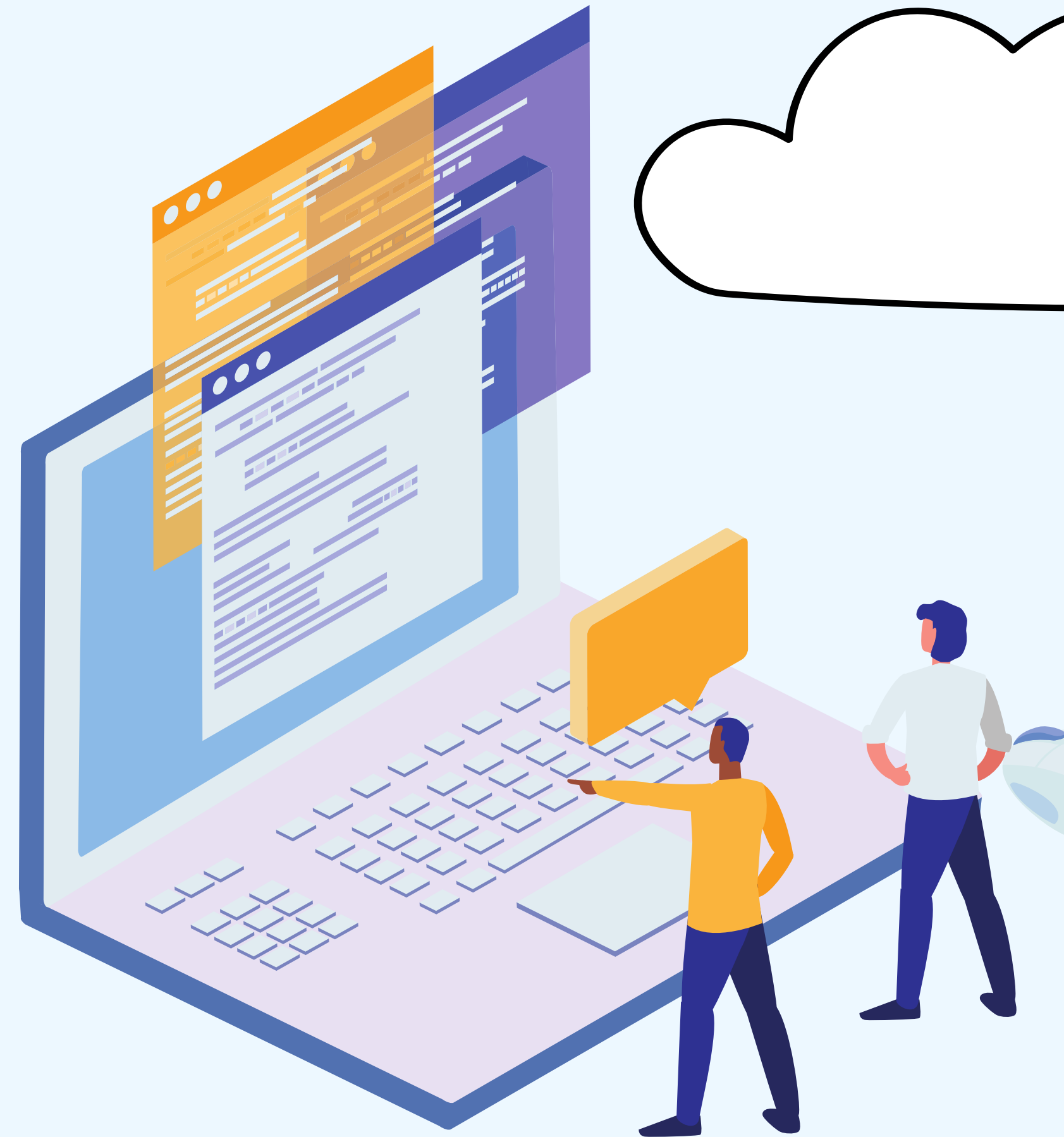


SOFTWARE ENGINEERING: A PRACTITIONER'S APPROACH, 7TH EDITION
CHAPTER 27

PROJECT SCHEDULING



WHY PROJECTS FALL BEHIND

ROOT CAUSES OF DELAYED SOFTWARE

**Unrealistic deadlines forced
by outside stakeholders**

**Scope creep & unreflected
requirement changes**

**Underestimation of required
effort or resources**

**Unrealistic deadlines forced
by outside stakeholders**

4-STEP NEGOTIATION STRATEGY

**Produce detailed historical
estimates**

**Design incremental delivery
plan**

**Demonstrate performance
gap clearly**

**Present functional tradeoff
choices**

SCHEDULING PRINCIPLES

1 **Compartmentalization**

Decompose the project into manageable activities, actions and distinct work tasks.

2 **Interdependency**

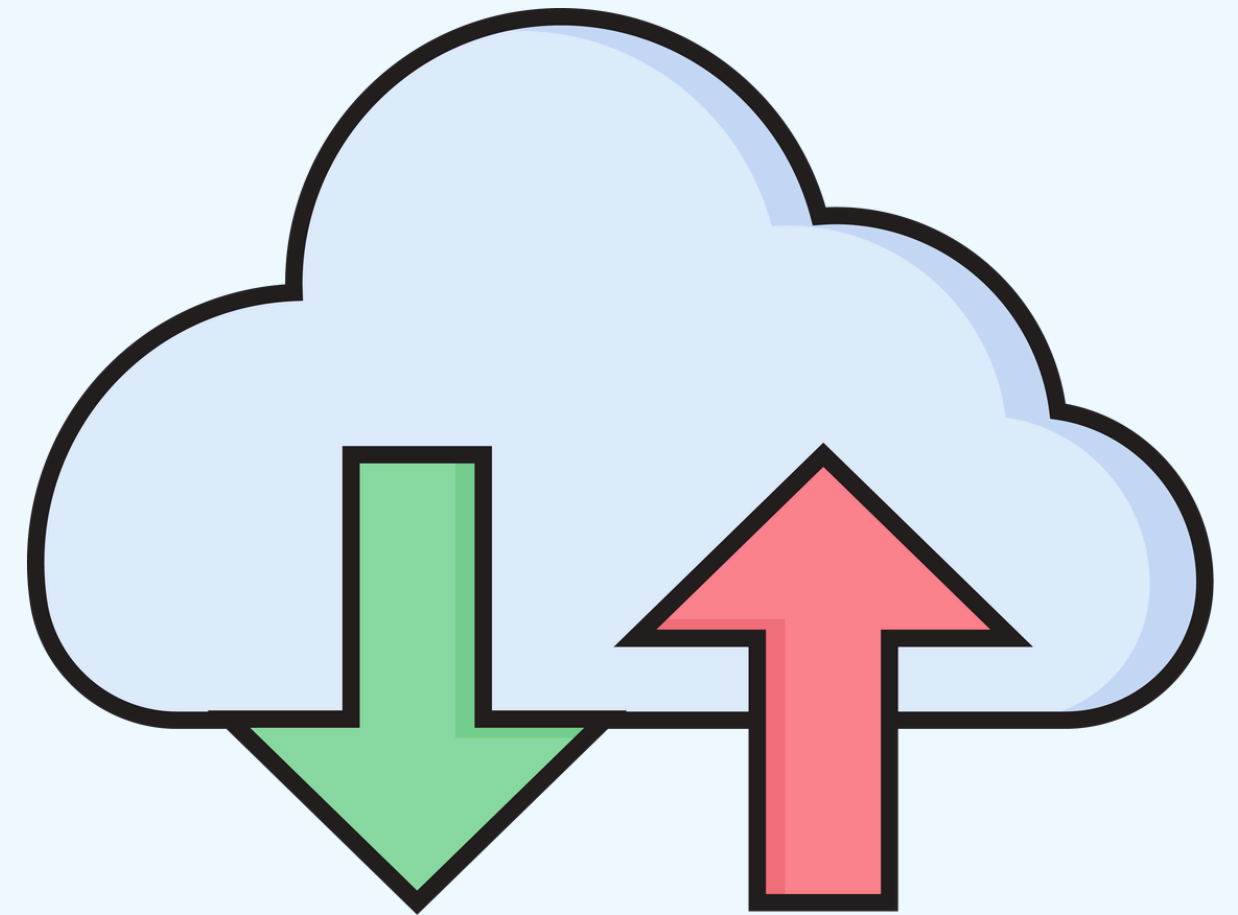
Determine sequential vs. parallel tasks to establish clear workflow dependencies.

3 **Time Allocation**

Assign work units (person-days), start dates, and completion dates to each task.

4 **Effort Validation**

Ensure scheduled concurrent tasks never exceed total available team staffing.



SCHEDULING PRINCIPLES

5 Defined Responsibilities

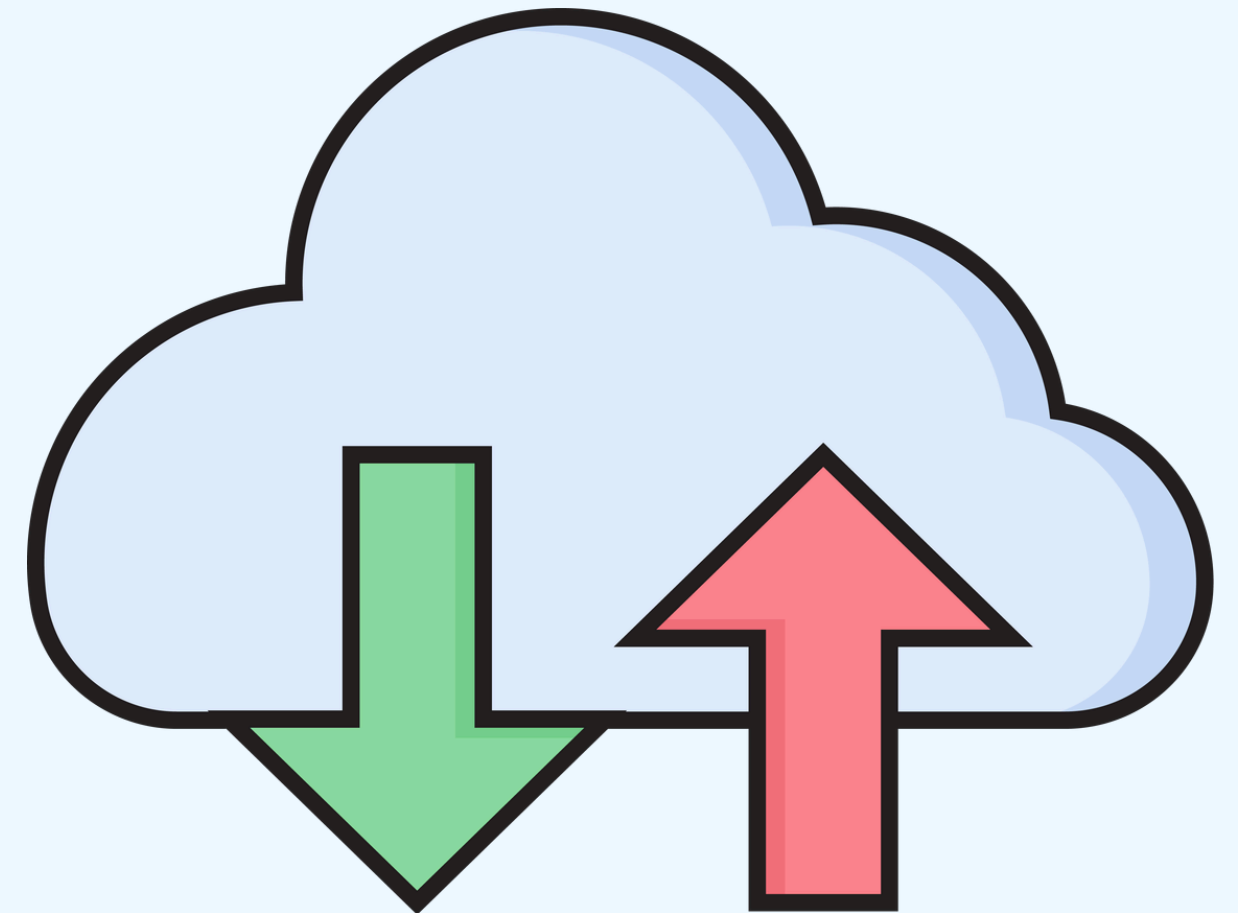
Assign every scheduled task to a specific, named team member.

6 Defined Outcomes

Ensure every scheduled task produces a specific work product or deliverable.

7 Defined Milestones

Attach tasks or task groups to formal milestones that are reviewed and approved for quality.

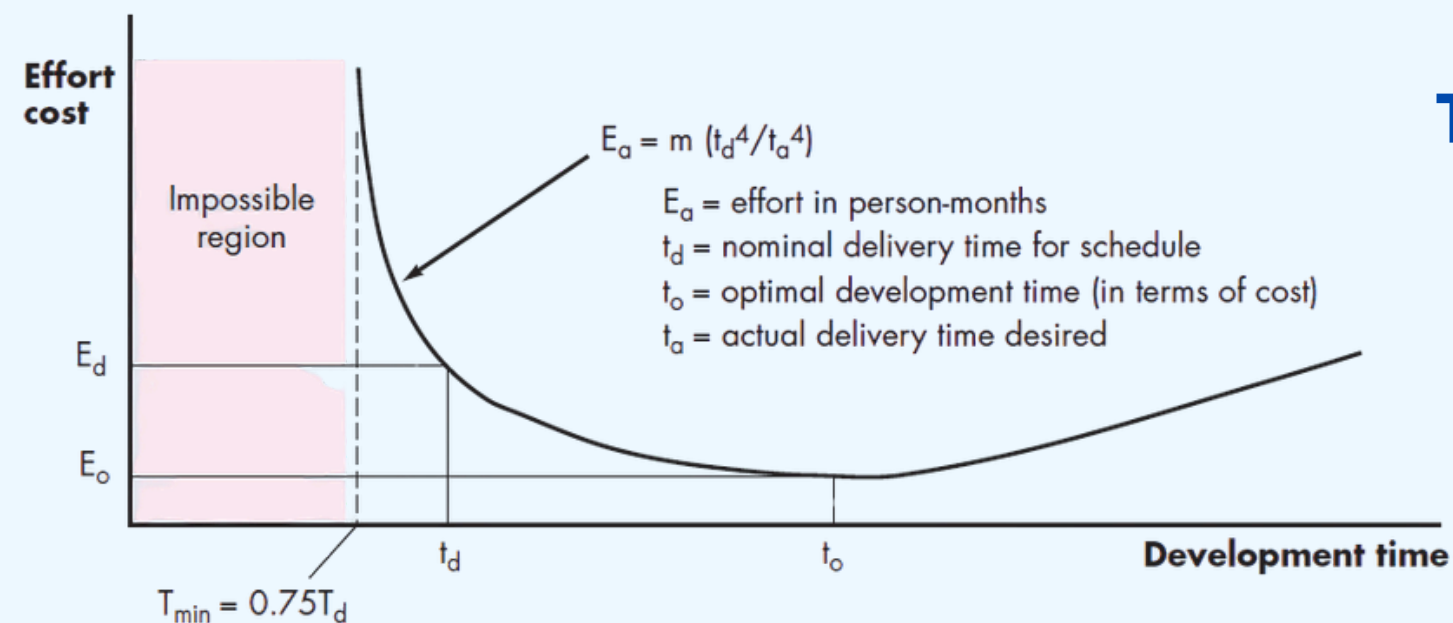


THE RELATIONSHIP BETWEEN PEOPLE AND EFFORT

Brooks' Law & The PNR Curve

"Adding manpower to a late software project makes it later"

- Communication overhead and ramp-up time eat up productivity



The Impossible Region

- Schedule compression beyond 0.75 t_d creates exponential risk of failure regardless of manpower added

THE SOFTWARE EQUATION

Software Life-Cycle Effort Equation:

$$E = \frac{L^3}{P^3 \cdot t^4}$$

Where: L = Lines of Code, P = Productivity Parameter, t = Time in years

The Benefit of Timeline Extension

- Because effort varies inversely with the 4th power of time (t^4), small increases in duration lead to dramatic savings in effort!

Practical Example

- Delivery in 1.3 years → Requires 8 people (12 person-years)
- Extend to 1.75 years → Requires only 4 people (3.8 person-years)

EFFORT DISTRIBUTION RULE

The classic guideline for software project effort distribution across the development lifecycle:

40%

**Front-end Analysis and
Design**

20%

**Coding /
Construction**

40%

**Back-end Testing &
Debugging**

Note: Agile methodologies and critical human-rated software adapt these baseline percentages based on project risks.

5 Common Project Types

Concept Development

New Application

**Application
Enhancement**

**Application
Maintenance**

Reengineering

TASK SETS & DECOMPOSITION

Refinement of Software Engineering Actions

Task definition: Action 1.1 Concept Scoping

1.1.1 Identify need, benefits and potential customers;

1.1.2 Define desired output/control and input events that drive the application;

Begin Task 1.1.2

1.1.2.1 TR: Review written description of need⁷

1.1.2.2 Derive a list of customer visible outputs/inputs

1.1.2.3 TR: Review outputs/inputs with customer and revise as required; endtask
Task 1.1.2

1.1.3 Define the functionality/behavior for each major function;

Begin Task 1.1.3

1.1.3.1 TR: Review output and input data objects derived in task 1.1.2;

1.1.3.2 Derive a model of functions/behaviors;

1.1.3.3 TR: Review functions/behaviors with customer and revise as required;
endtask Task 1.1.3

1.1.4 Isolate those elements of the technology to be implemented in software;

1.1.5 Research availability of existing software;

1.1.6 Define technical feasibility;

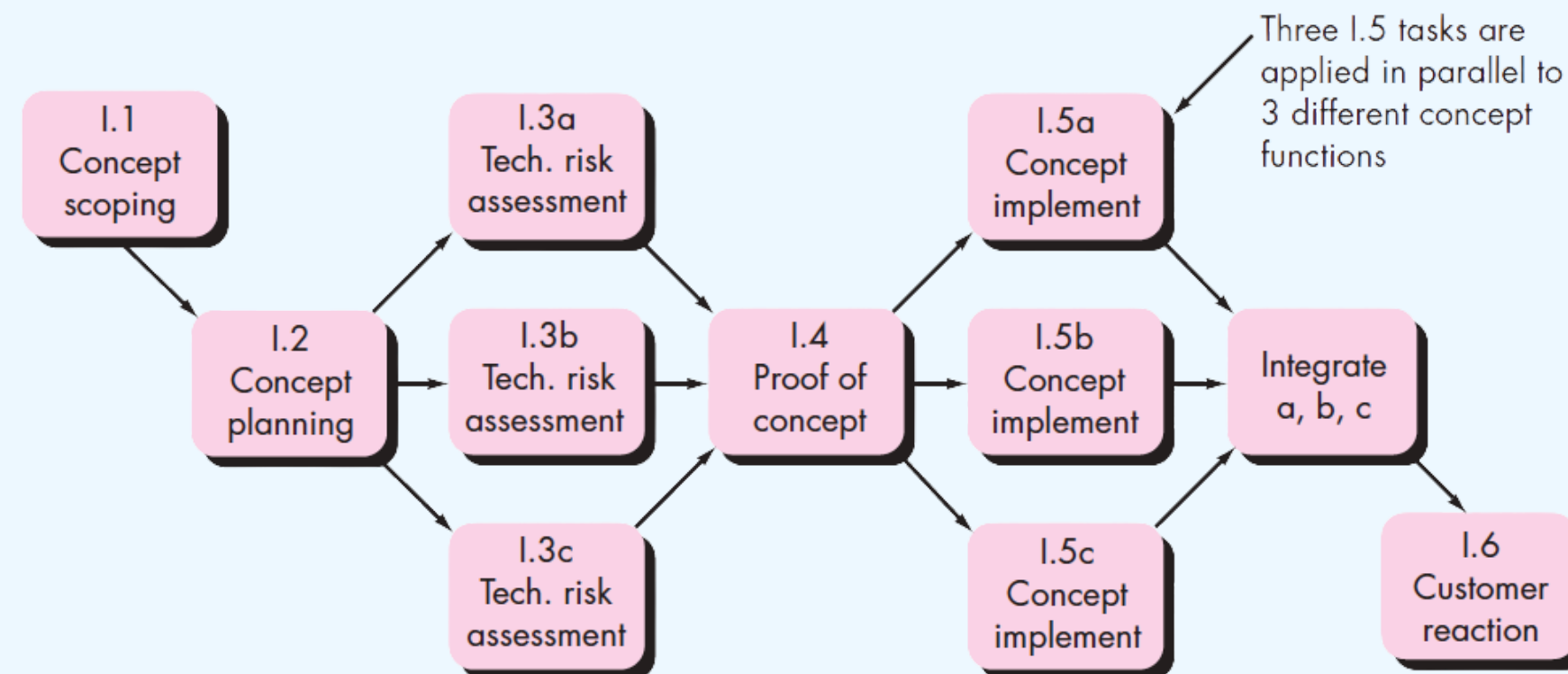
1.1.7 Make quick estimate of size;

1.1.8 Create a scope definition;

endtask definition: Action 1.1

TASK NETWORK & CRITICAL PATH

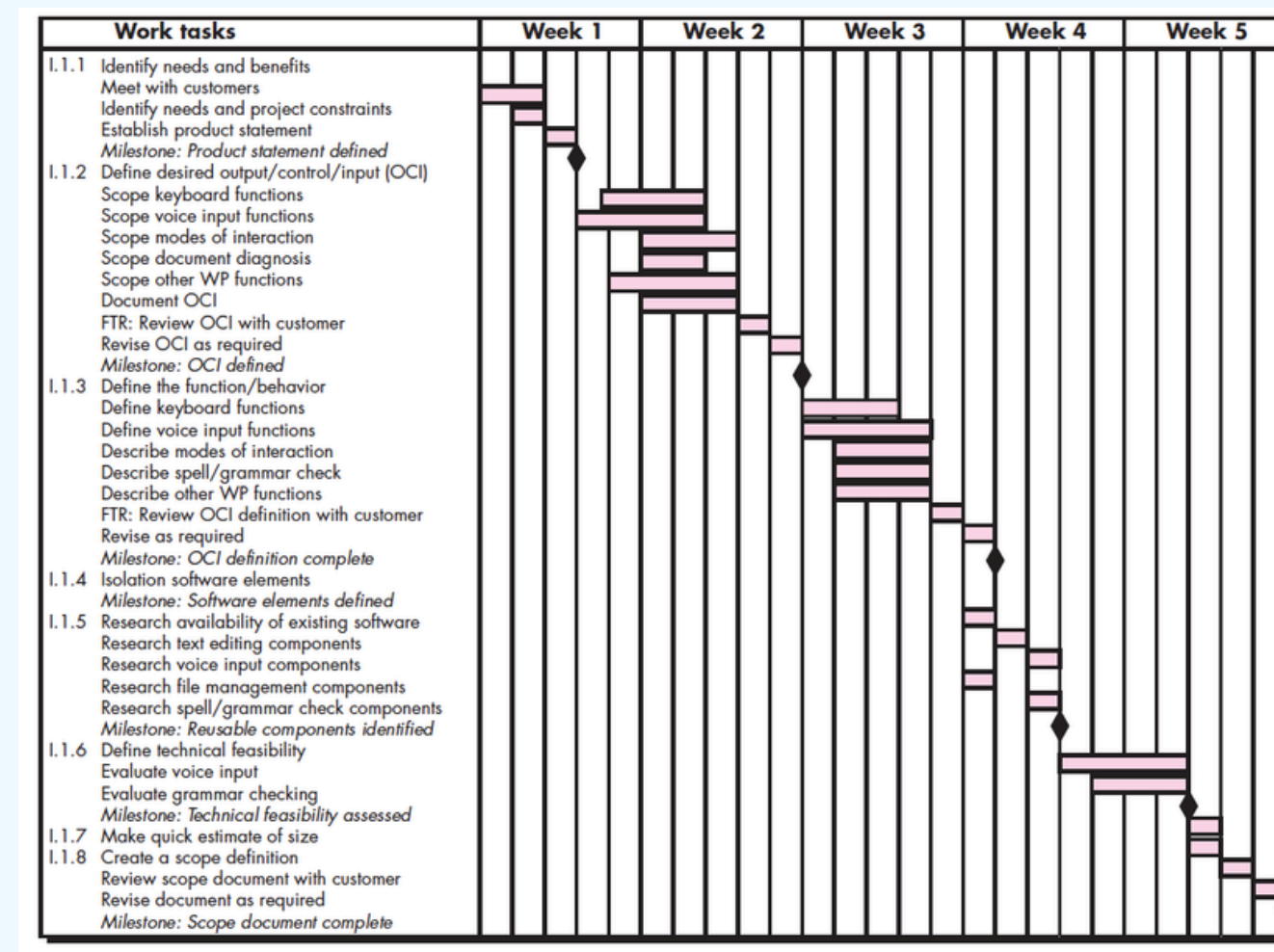
A Task Network (Activity Network) graphically represents task flow, parallel execution, and interdependencies.



Critical Path Rule: The longest sequence of dependent tasks determines the minimum project duration. Any delay on critical path tasks directly delays project completion!

GANTT CHARTS & PROJECT TABLES

Work breakdown structures are mapped into calendar schedules using Time-Line (Gantt) Charts and Project Tables.



TRACKING & PROJECT CONTROL

HOW MANAGERS TRACK PROGRESS

**Periodic status meetings
with technical staff**

**Evaluating software
technical review outcomes**

**Comparing completed
milestones vs. schedule**

**Comparing actual vs.
planned start/finish dates**

**Informal daily check-ins
with team members**

EARNED VALUE ANALYSIS

**BCWS: Budgeted Cost of
Work Scheduled**

**BCWP: Budgeted Cost of
Work Performed**

**Schedule Variance (SV):
 $BCWP - BCWS$**

**Schedule Performance
Index (SPI): $BCWP / BCWS$**



KEY TAKEAWAYS

Schedule Discipline

- Late delivery stems from poor scheduling and unmanaged deadline pressures.

Manage People & Effort

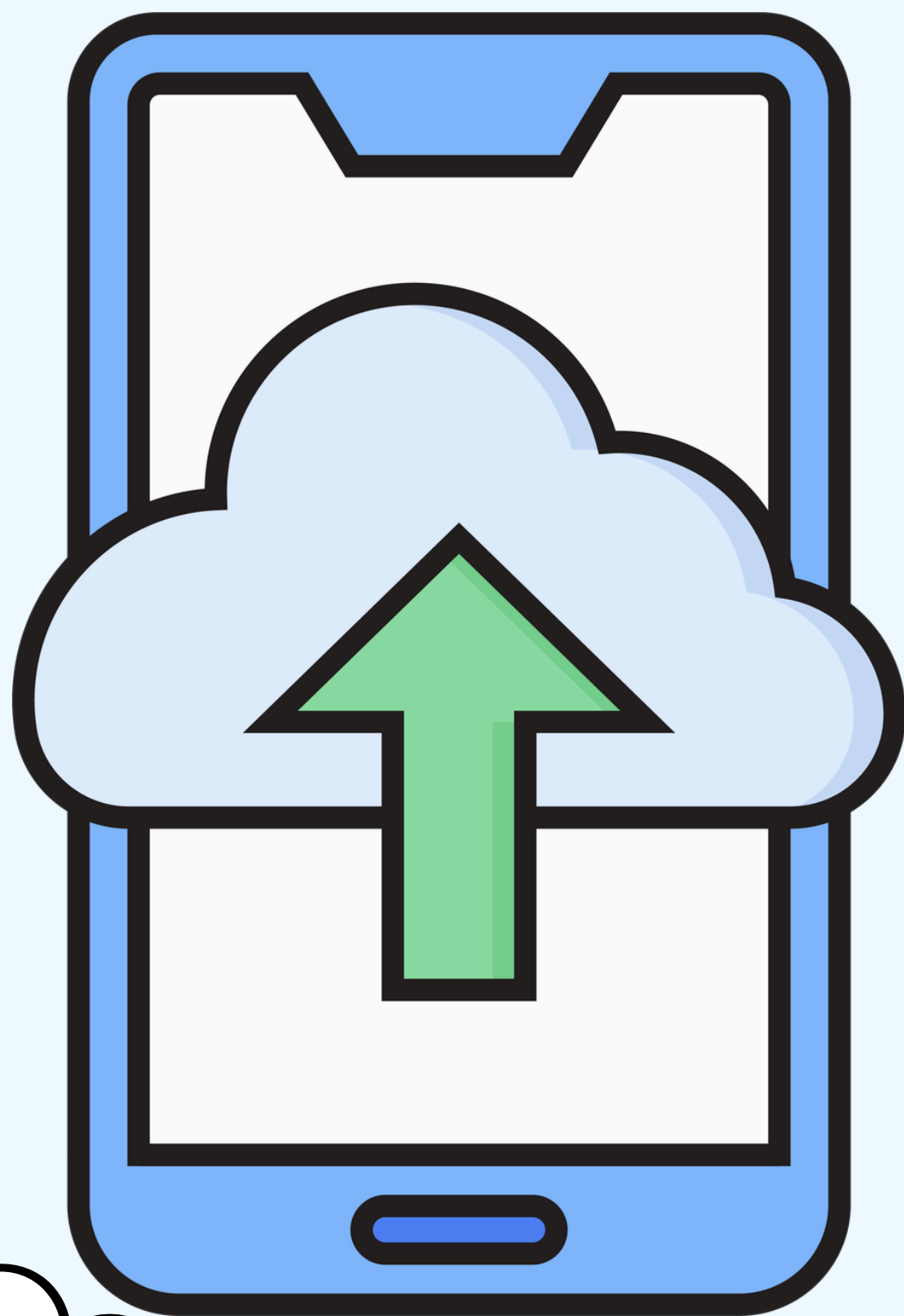
- Compress schedule only to limits ($0.75t_d$) to avoid the impossible region.

Follow 40-20-40 Rule

- Invest heavily in front-end design and back-end testing over raw coding.

Track Continuously

- Monitor task networks, critical paths, and milestones "one day at a time".



THANK YOU

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

